



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
Faculty of Electrical Engineering, Automatics, Computer Science and Biomedical Engineering

Engineering Thesis

Enhancing Small Language Models via Chain-of-Thought Distillation
Ulepszanie małych modeli językowych poprzez destylację łańcucha
myślowego

Author:	<i>Kacper Rabczewski</i>
Degree programme:	<i>Computer science and intelligent systems</i>
Supervisor:	<i>Dr. Krzysztof Kluza</i>

Kraków, 2025

Put your acknowledgements here

Abstract

This thesis investigates whether supervised chain-of-thought distillation (SCoTD) can measurably improve the reasoning accuracy of a small language model on math word problems. A strong teacher model generates structured reasoning traces (CoT) for GSM8K, which are cleaned and deduplicated, and then used to finetune a 3B-class student via parameter-efficient adapters (LoRA) under 4-bit quantization. The training stack relies on modern open tooling for reproducibility and efficient use of a single 24GB GPU. Evaluation uses strict numeric exact-match parsing on GSM8K.

Results show that SCoTD improves the student over a non-finetuned base with CoT prompting (78.54% vs 70.51%). A label-only variant also improves over its baseline (15.01% vs 10.99%), though strict formatting requirements likely undercount true ability. The teacher reaches 93.61% on the GSM8K test split. Contributions include a cost-aware CoT data generation pipeline, a reproducible finetuning setup (LoRA + 4-bit nf4 + bf16), and an empirical analysis of accuracy, training dynamics, and operational costs. These findings support SCoTD as a practical method to enhance small models' reasoning under resource constraints.

Keywords: chain-of-thought, distillation, LoRA, QLoRA, GSM8K, small language models, reasoning, quantization

Contents

1. Introduction	3
2. Background	7
2.1. Large language models	7
2.1.1. Objective and pretraining	7
2.1.2. Transformer	7
2.1.3. Decoder-only LLMs	8
2.1.4. Scaling laws	9
2.1.5. Limitations and small-model challenges	9
2.2. Prompting	10
2.2.1. Paradigms	10
2.3. Knowledge distillation	10
2.3.1. Objectives and loss design	10
2.3.2. Distillation for sequence models and LLMs	11
2.3.3. Distilling reasoning	11
2.3.4. When to expect gains	11
2.4. Parameter-efficient fine-tuning (PEFT)	12
2.4.1. Method families	12
2.4.2. Quantization-aware PEFT	12
2.5. Tokenizers	13
2.5.1. Subword methods	14
2.5.2. Special tokens and sequence framing	15
2.6. GSM8K	16
2.6.1. Dataset structure	16

1. Introduction

Large language models (LLMs) have achieved strong performance on a wide range of language tasks. However, small models (sub-4B parameters) often fall short on multi-step reasoning benchmarks such as GSM8K, where solutions require decomposing a problem into intermediate steps and computing a precise numeric answer [1]. Chain-of-thought (CoT) prompting can elicit step-by-step reasoning from strong teachers and improve accuracy [2], especially when combined with self-consistency sampling [3]. Yet, relying on large models at inference time is costly and slow. This motivates distilling teacher reasoning traces into a small student that can run efficiently on commodity hardware.

This thesis studies supervised chain-of-thought distillation (SCoTD): a strong teacher generates structured CoTs for GSM8K questions; a small student is finetuned to reproduce these traces and final answers. The student is trained with parameter-efficient finetuning (LoRA) [4] on quantized weights (QLoRA) [5], enabling training on a single 24GB GPU while preserving headroom to learn reasoning behavior. The approach uses standard open-source tooling for models and tokenization [6, 7, 8].

Problem and motivation

Small models are attractive due to their lower latency, footprint, and cost, but they struggle with multi-step math reasoning under greedy decoding. CoT distillation promises to transfer the teacher’s intermediate reasoning patterns, potentially improving student accuracy without increasing inference-time cost. The practical question is whether a 3B-class model can benefit sufficiently from SCoTD to close part of the gap to teacher performance on GSM8K.

Research questions

- RQ1: Does SCoTD improve a 3B-class model on GSM8K compared to the non-finetuned base under CoT prompting?
- RQ2: How does label-only supervised finetuning compare to SCoTD and base baselines under strict numeric exact-match evaluation?

Scope

The study focuses on a single small decoder-only model (Qwen2.5-3B) [9] and a single dataset (GSM8K) [1]. Teacher CoTs are produced via an API-accessible, strong reasoning model configured to emit structured steps and a line-form final answer. The student is trained with LoRA on 4-bit quantized base weights using bf16 compute, gradient checkpointing, and standard optimizers. Evaluation uses greedy decoding with strict numeric parsing; outputs that deviate from the expected format are counted as incorrect.

Contributions

- A practical, cost-aware pipeline for generating and cleaning teacher CoTs for GSM8K, with concurrency-tuned API calls and per-sample metadata for analysis.
- A reproducible PEFT training setup (LoRA + 4-bit nf4 + bf16) that fits on a single 24GB GPU while targeting attention and MLP projection modules.
- Empirical evidence that SCoTD improves a 3B-class model over base CoT prompting on GSM8K (78.54% vs 70.51%), with a strong teacher at 93.61% and a label-only student that improves over baseline (15.01% vs 10.99%) despite strict-format penalties.
- Operational notes on prompt design, cleaning/deduplication, truncation policy, and evaluation pitfalls that affect measured accuracy.

Method overview

The pipeline comprises teacher CoT generation, cleaning and deduplication, dataset processing, student finetuning (SCoTD and label-only variants), and evaluation. The student relies on PEFT with LoRA adapters [4] over a quantized backbone [5]. Tokenization is handled with a fast subword tokenizer [7, 8]. Inference uses greedy decoding and a strict numeric exact-match metric. The software stack builds on Transformers and related tooling [6]. The teacher is a strong reasoning model (DeepSeek-R1 Distill [10]) accessed via OpenRouter [11]. Figure 1.1 summarizes the end-to-end pipeline and artifact flow.

Results summary

SCoTD yields a clear gain over the base model under CoT prompting (78.54% vs 70.51%). Label-only finetuning improves over its baseline (15.01% vs 10.99%), though strict formatting likely undercounts correct numeric computations. The teacher achieves 93.61% on GSM8K test, providing headroom and confirming the value of distilling high-quality reasoning traces.

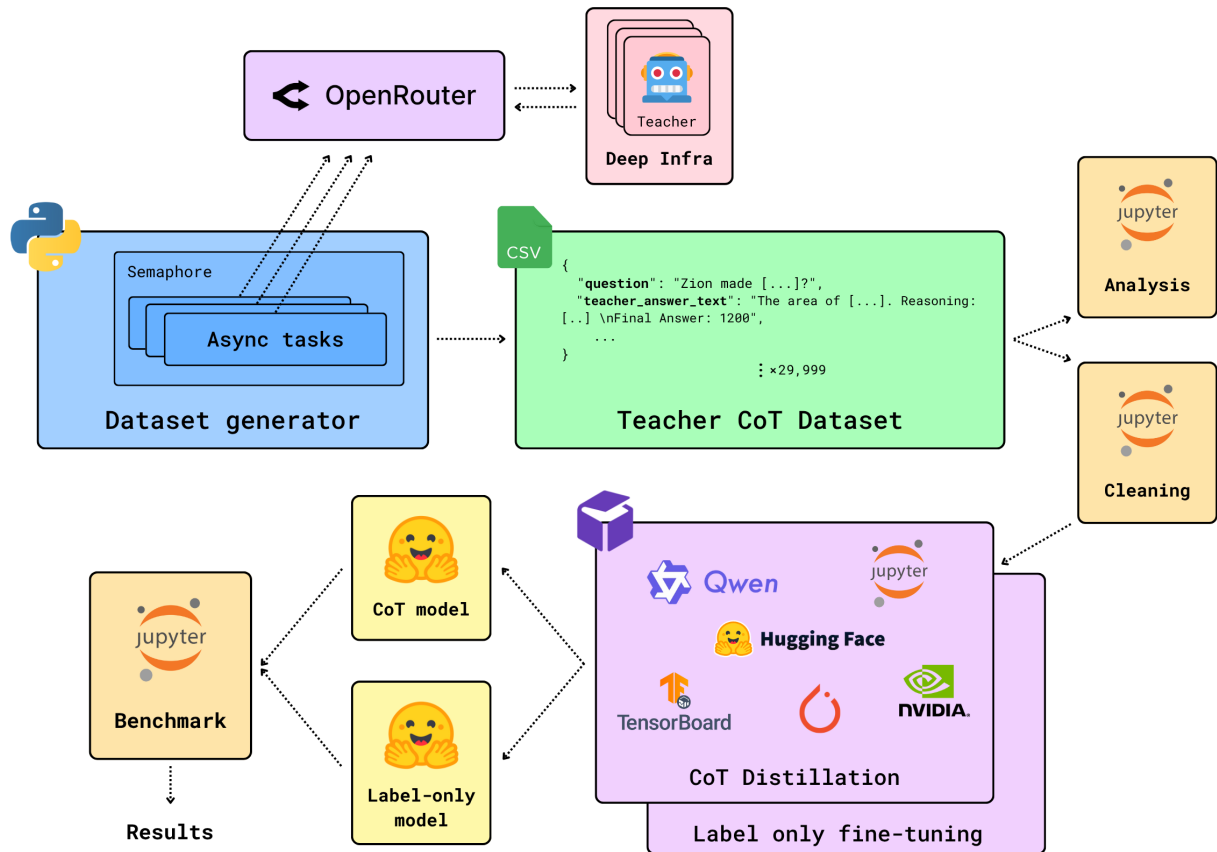


Figure 1.1. End-to-end SCoTD pipeline.

Author background and motivation

The author is a Co-Founder at an AI phishing detection startup that uses LLMs to analyze URLs and webpage content for malicious indicators, and a Machine Learning Engineer at OneRail building LLM-powered support workflows for same-day logistics. These roles highlighted the need for reliable, cost-efficient reasoning under tight latency and single-GPU constraints, motivating SCoTD as a practical path to higher accuracy.

2. Background

2.1. Large language models

Large language models (LLMs) lie at the core of the current AI revolution. They are parametric probabilistic models that learn to represent the distribution of natural language sequences and generate text by autoregressively predicting one token at a time. Formally, given a tokenized sequence $(x_1, \dots, x_T)$, an LLM models $p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{<t})$ and is trained to minimize the cross-entropy between predicted and observed tokens. LLMs derive their broad linguistic and factual knowledge from pretraining on massive text corpora using this objective, which at scale enables emergent capabilities such as in-context learning and reasoning.

2.1.1. Objective and pretraining

The standard objective is a next-token prediction:

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}), \quad (2.1)$$

optimized over large, diverse datasets using gradient descent. Conceptually, this process can be understood as a vast, GPU-driven lossy compression - the model distills the statistical regularities and patterns of human language into a compact, parameterized form that enables general-purpose text understanding and generation. While pretraining is purely self-supervised, downstream adaptation may use supervised fine-tuning or preference-based methods to align behavior.

2.1.2. Transformer

Transformer architecture [12] underlies most modern LLMs. The main building blocks are:

- **Embeddings and positional encoding:** Tokens are mapped to dense vectors and augmented with position information, either sinusoidal or learned, to make the model order-aware.
- **Multi-head self-attention:** For each position, queries Q , keys K , and values V are computed by linear projections of the hidden states. Scaled dot-product attention aggregates information across positions:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V. \quad (2.2)$$

Multiple heads attend in parallel to different subspaces, and their outputs are concatenated and projected.

- **Position-wise feed-forward network (FFN):** A two-layer MLP with a nonlinearity (e.g., GELU) is applied independently to each position.
- **Residual connections and layer normalization:** Each sublayer is wrapped with residual additions and normalization for stability.

Computationally, attention has $O(L^2d)$ time and $O(L^2)$ memory in sequence length L (per batch and head factors omitted), while FFNs dominate parameter count with $O(d^2)$.

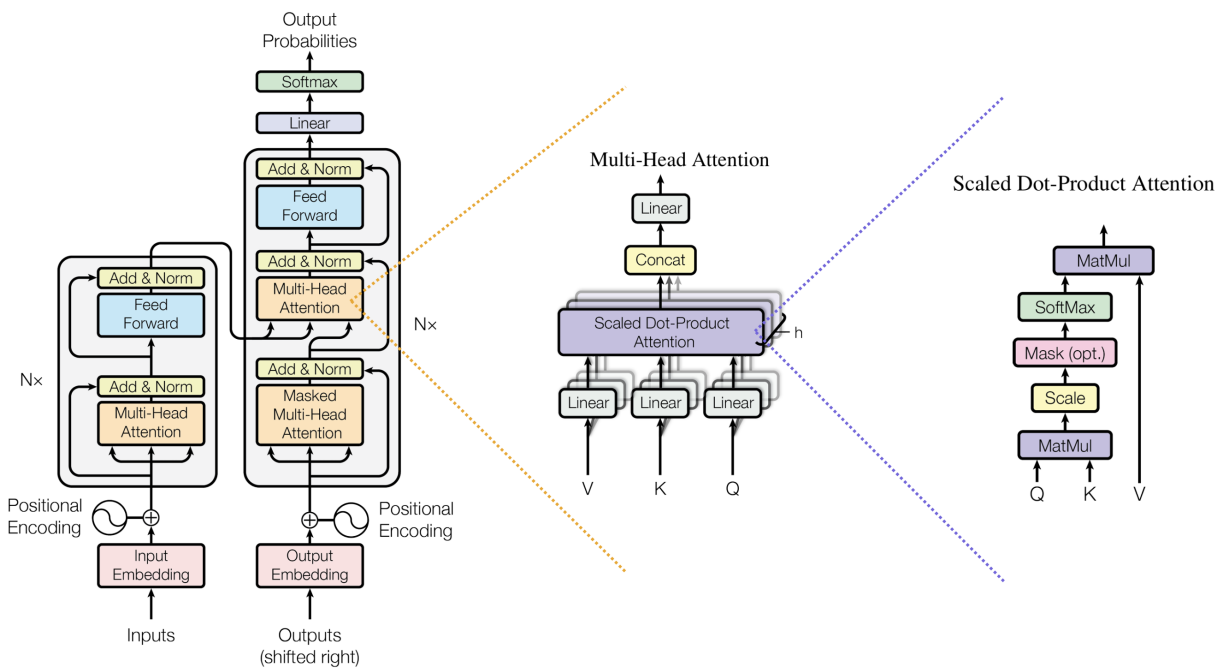


Figure 2.1. Transformer encoder-decoder block [12].

2.1.3. Decoder-only LLMs

Decoder-only LMs use a lower-triangular mask in self-attention so that token t attends only to positions $< t$. This enforces autoregressive factorization and enables left-to-right generation. During inference, key-value (KV) caching stores past projections to avoid recomputation, reducing per-token latency from quadratic to approximately linear in L for attention, with constant-depth projections per layer.

2.1.4. Scaling laws

Transformers scale predictably with model size (parameters), data (tokens), and compute. Empirically, test loss follows power-law scaling with these resources [13], implying steady gains as capacity and data increase. Practical costs rise superlinearly with size due to:

- **Training:** attention’s $O(L^2)$ complexity amplifies FLOPs and memory.
- **Inference:** latency and memory scale with depth and width; context length increases multiply attention cost without specialized kernels.

Despite costs, scaling yields emergent capabilities such as in-context learning and improved generalization [14].

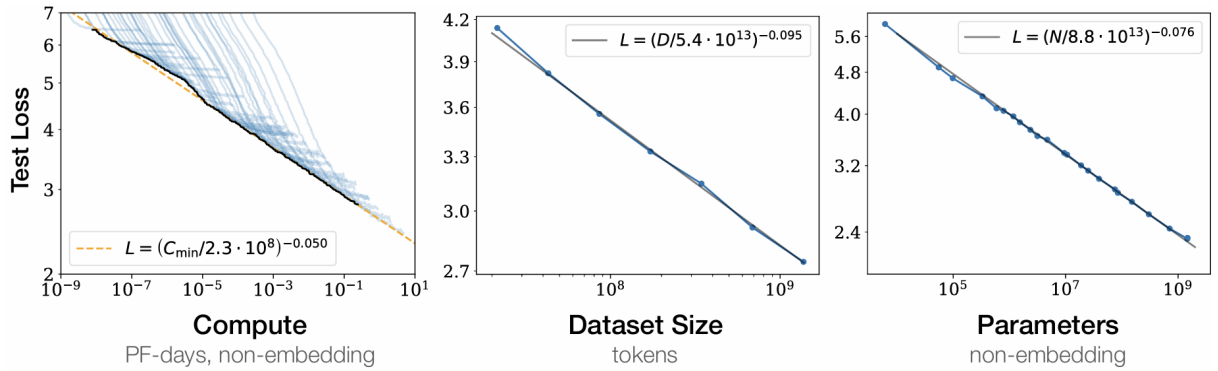


Figure 2.2. Scaling laws: test loss vs. effective compute/model size/data on logarithmic axes [13].

2.1.5. Limitations and small-model challenges

Large language models are known to have several limitations including:

- **Hallucinations:** the model may produce fluent but unsupported statements.
- **Context and computation limits:** finite context windows and lack of external working memory hinder long-horizon tasks.
- **Compositionality:** chaining steps and maintaining intermediate variables is difficult under greedy decoding.

Small models (e.g., <4B parameters) face additional constraints which degrade multi-step arithmetic and logical reasoning, especially when strict output formats are required. While prompting and decoding strategies can mitigate some issues, structural capacity often remains the bottleneck without additional supervision or architectural support.

2.2. Prompting

Prompting is a practice of specifying a model’s task through natural language. It functions as a programming interface to language models. Instruction tuning and alignment methods improve adherence to prompts [15].

2.2.1. Paradigms

Several prompting paradigms have emerged, often combined in practice:

- **Zero-shot prompting:** The model receives only task instructions and the input. This tests inherent generalization from pretraining and any instruction tuning. It is simple and cheap but sensitive to wording and often weaker on tasks requiring intermediate computation.
- **Few-shot prompting:** The prompt includes k input-output examples that implicitly define the task and desired format, enabling in-context learning [14].
- **Chain-of-Thought (CoT) prompting:** The prompt requests step-by-step reasoning, eliciting intermediate rationales before the final answer [2]. CoT can improve complex, multi-step tasks like arithmetic by encouraging deliberate decomposition, but increases latency and token usage.

2.3. Knowledge distillation

Knowledge distillation (KD) transfers the behavior of a high-capacity teacher model to a typically smaller student [16]. The core idea is that softened target distributions carry "dark knowledge" about class or token similarities that hard labels do not expose, often improving generalization while enabling compression and faster inference.

2.3.1. Objectives and loss design

Response-based KD is the most widely used approach. In this setting, the student is trained to match the teacher's output distribution. Let $z^{(T)}$ and $z^{(S)}$ be teacher and student logits, and $T > 0$ a temperature. The softened distributions are $p^{(T)} = \text{softmax}(z^{(T)}/T)$ and $p^{(S)} = \text{softmax}(z^{(S)}/T)$. A common loss combines distillation with standard supervision:

$$\mathcal{L} = (1 - \alpha) \text{CE}(y, \text{softmax}(z^{(S)})) + \alpha T^2 \text{KL}(p^{(T)} \| p^{(S)}), \quad (2.3)$$

where y are hard labels and $\alpha \in [0, 1]$ balances terms; the T^2 factor compensates for gradient scaling [16].

Beyond responses, feature-based KD aligns intermediate representations (e.g., hint layers in FitNets [romero2015fitnets] or attention maps in Attention Transfer [17]), while relation-based KD matches instance relations (e.g., pairwise similarities). Self-distillation trains a student from its own or a previous generation's predictions (Born-Again Networks [18]); data-augmented KD uses pseudo-labels on unlabeled data (Noisy Student [19]). See [20] for a survey.

2.3.2. Distillation for sequence models and LLMs

In language modeling, KD is applied at either:

- **Token level:** match teacher logits or probabilities at each decoding step, often alongside cross-entropy to the ground truth.

- **Sequence level:** train on sequences decoded by the teacher (e.g., beam search), simplifying the target distribution and reducing mode entropy [21].

Both offline (fixed teacher) and online (co-trained or periodically refreshed teacher) regimes exist. For autoregressive models, exposure bias remains; KD reduces variance but does not remove the mismatch between training and generation. Practical choices include which layers to distill, how to weight losses over time, and whether to distill only on high-confidence tokens.

2.3.3. Distilling reasoning

Symbolic Chain-of-Thought Distillation (SCoTD) transfers step-by-step reasoning from a large teacher to a smaller student by training on teacher-generated rationales rather than logits [22]. Concretely: (i) curate a few-shot CoT prompt set; (ii) for each unlabeled input x , sample multiple CoT rationales z and a prediction y from the teacher; (iii) optionally filter samples to keep only label-correct pairs when gold labels are available; and (iv) fine-tune the student with the standard token-by-token language modeling objective to generate the concatenated rationale and answer. At inference, the student can decode a single rationale+answer greedily or sample multiple reasoning paths and select the majority answer via self-consistency [3].

Empirically, SCoTD enables 125M-1.3B students to "think step-by-step" and improves accuracy on commonsense QA benchmarks (CSQA, QuaRel, OpenBookQA). A key finding is that oversampling rationales per input (e.g., ~ 30 CoTs/example) is more important than aggressive filtering. Self-consistency particularly helps when training used noisy, unfiltered CoTs (few-shot setting), whereas its benefit diminishes once supervised filtering removes incorrect teacher answers.

2.3.4. When to expect gains

KD is most effective when the teacher's distribution contains informative structure that the student cannot infer from hard labels alone. For CoT, expected gains increase with teacher rationale quality and student capacity to model longer sequences.

2.4. Parameter-efficient fine-tuning (PEFT)

Full fine-tuning updates all model parameters and optimizer states, which scales memory, compute, and communication roughly linearly with parameter count and often requires multi-GPU setups. Optimizers like Adam maintain first and second moments, tripling memory relative to parameters alone. Checkpointing, gradient storage, and activation memory further increase requirements. Parameter-efficient fine-tuning (PEFT) alleviates these costs by freezing the backbone and training a small number of additional parameters that modulate the forward pass. This also improves modularity (task-specific heads), reduces overfitting, and enables fast switching between tasks.

2.4.1. Method families

Several PEFT families differ in where trainable parameters are introduced and how they influence the network:

- **Adapters:** Small bottleneck modules (down-projection, nonlinearity, up-projection) are inserted within Transformer blocks and trained while the base model is frozen [23]. They achieve near full fine-tuning performance with orders of magnitude fewer trainable parameters and support composition across tasks.
- **Prompt and prefix tuning:** Instead of modifying internal weights, these methods learn task-specific soft prompts. Prompt tuning optimizes a small set of embeddings prepended to the input [24], while prefix-tuning optimizes continuous key/value prefixes injected into attention at each layer [25]. These approaches are extremely parameter-efficient and work well when the pretrained model already encodes the needed capabilities; performance can depend on model scale and task complexity.
- **Low-rank adaptation (LoRA):** LoRA freezes the original weight matrix W and parameterizes the update as a low-rank decomposition $\Delta W = AB$ with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$ [4]. A common scaled form is

$$W' = W + \frac{\alpha}{r} AB, \quad (2.4)$$

where $A \in \mathbb{R}^{d_{\text{out}} \times r}$ and $B \in \mathbb{R}^{r \times d_{\text{in}}}$ are the only trainable parameters and α controls the update magnitude. LoRA is typically applied to attention and MLP projection matrices. Benefits include: (i) minimal trainable parameters, (ii) negligible inference overhead when merged into W , and (iii) modular adapters that can be stored and shared independently of the base model.

2.4.2. Quantization-aware PEFT

Quantization-aware fine-tuning combines PEFT with low-precision base weights to further reduce memory. QLoRA keeps the frozen backbone in 4-bit quantization (e.g., NF4) with per-channel scaling and "double quantization" to compress scales, while performing forward/backward compute in higher precision (e.g., bf16) and training only the PEFT parameters [5]. In this setup:

- The quantized backbone reduces memory footprint without updating full-precision weights.
- LoRA (or other PEFT parameters) are maintained in higher precision and receive gradients.
- Careful quantizer design (NF4) and optimizer choices preserve training stability and accuracy close to full-precision baselines.

This approach enables fine-tuning larger backbones under tight memory budgets and decouples adaptation capacity (set by PEFT parameters like LoRA rank) from the storage format of the base model. Practical backends for 4-bit inference/training are widely available [26].

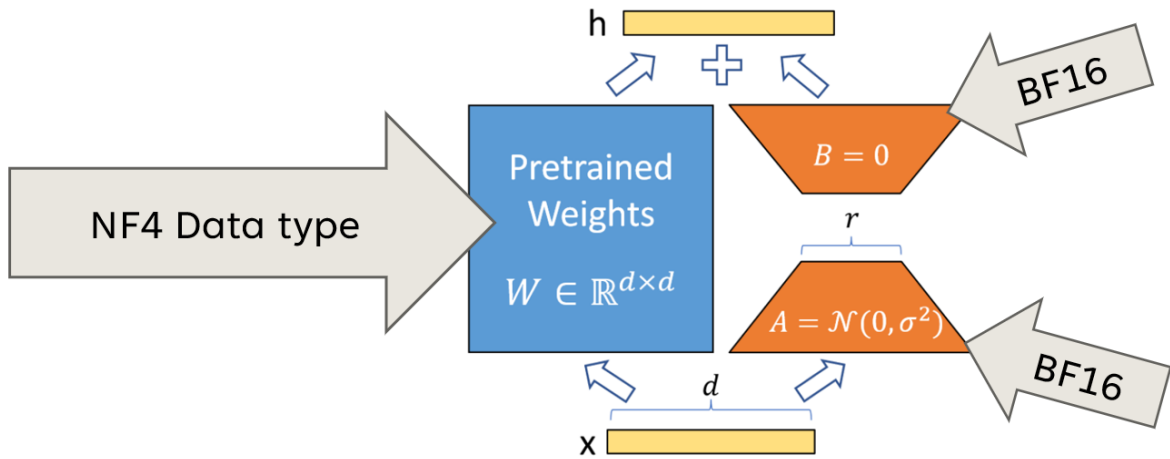


Figure 2.3. Model parameters under QLoRA NF4 + LoRA: frozen backbone in 4-bit quantization with per-channel scales; trainable LoRA parameters in higher precision. Forward/backward compute in bf16 [27].

2.5. Tokenizers

Tokenization converts raw text into a sequence of discrete token IDs drawn from a fixed vocabulary. Language models as one of the first forward pass steps consume sequence of token IDs as input and map them to their embeddings vectors. A good scheme balances coverage (few unknowns), compact sequences (shorter length for a given text), and stability of encoding/decoding.

2.5.1. Subword methods

Pure word-level vocabularies suffer from out-of-vocabulary (OOV) issues, while character-level tokenization produces very long sequences. Subword tokenization offers a compromise by decomposing words into frequent subunits that compose to any string, including rare words and misspellings [8].

Common approaches:

- **Byte-Pair Encoding (BPE)**: the most widely used method in modern LLMs, starts from a large text corpus and iteratively merges the most frequent adjacent pairs to form larger subwords until a target vocabulary size is reached [8]. BPE yields deterministic segmentations given the learned merges and has no explicit probabilistic model.
- **WordPiece**: subword units to maximize likelihood of a training corpus, typically under a simple probabilistic model over segmentations; greedy decoding selects the best segmentation. It typically includes reserved continuation markers to indicate that a subword is not word-initial.
- **SentencePiece**: optimizes a probabilistic unigram model over a candidate subword inventory and prunes units that hurt likelihood [7]. It supports sampling alternative segmentations during training for robustness and can operate directly on raw text without external pre-tokenization.

Implementation frameworks such as SentencePiece and Transformers provide standardized training, encoding, and decoding pipelines [7, 6]. Variants like byte-level BPE ensure coverage of arbitrary Unicode by operating on UTF-8 bytes, which removes true OOVs at the cost of sometimes longer sequences for non-Latin scripts.

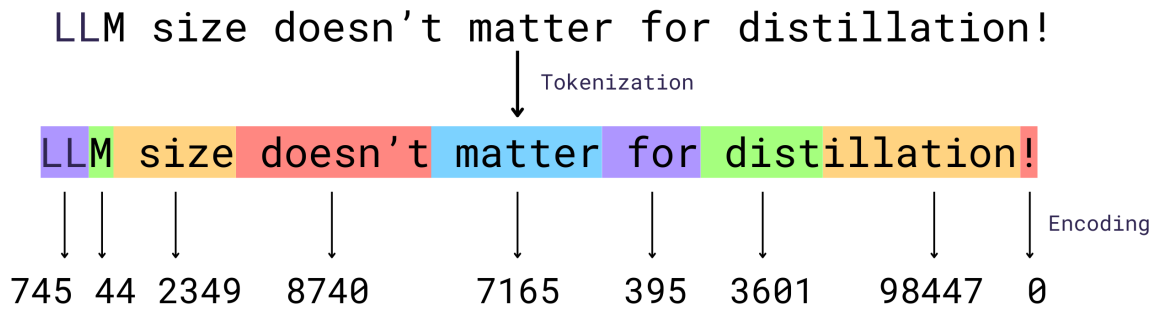


Figure 2.4. Tokenization example for OpenAI GPT-4o Tokenizer.

2.5.1.1. Vocabulary

Larger token vocabularies can represent up to whole words, reducing sequence length and attention cost. However, they implicitly increase embedding and output layer sizes, memory, and may cause overfitting. Smaller vocabularies generalize better to rare forms but inflate sequence length, which increases quadratic attention cost and risk of truncation. Practical choices range from 16k-200k subwords.

2.5.1.2. Normalization and pre-tokenization.

Many pipelines normalize text (e.g., Unicode NFKC, lowercasing, whitespace handling) to improve robustness; SentencePiece can integrate normalization and avoid external word boundary rules [7]. Pre-tokenizers may split on whitespace and punctuation to enforce word boundary awareness before applying subword merges. Choices here affect how punctuation, casing, and whitespace survive round-trip detokenization.

2.5.2. Special tokens and sequence framing

Special tokens mark structure and control decoding:

- **BOS/EOS:** Begin-of-sequence and end-of-sequence delimit the sequence and signal termination.
- **PAD:** Padding aligns sequences within a batch to a common length; attention masks ensure pads are ignored during loss computation and attention.
- **Instruction tokens:** Special markers that indicate task boundaries or specify desired behavior (e.g., `<|user|>`, `<|assistant|>`, `<|thought|>`).

<|bos|>

<|user|>

You are given a list of numbers: [3, 1, 4, 1, 5].
Please compute their sum and provide the result.

<|assistant|>

The sum of [3, 1, 4, 1, 5] is 14.

<|eos|>

Figure 2.5. Example of conversation-formatted instruction-response prompt.

2.6. GSM8K

GSM8K is a benchmark of 8.5K English grade-school math word problems designed to evaluate multi-step reasoning in natural language [1]. Problems typically require 2-8 steps of elementary arithmetic (+, −, ×, ÷) and include human-written, natural-language solutions with inline “calculator annotations” (intermediate computations) and a final integer answer on a separate line. The benchmark is widely used to study prompting and reasoning supervision; standard scoring is exact numeric match, often evaluated under chain-of-thought prompting and self-consistency sampling [2, 3].

2.6.1. Dataset structure

Released in English via HuggingFace Datasets [28], GSM8K provides two configurations: a main set (question, rationale, final answer) and a socratic variant that interleaves sub-questions within the solution. Official splits comprise 7,473 training and 1,319 test problems [1]; each instance pairs a question string with a multi-step natural-language solution containing intermediate calculations and a single integer final answer.

Problem

Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?

Solution

Natalia sold $48/2 = <<48/2=24>>24$ clips in May. Natalia sold $48+24 = <<48+24=72>>72$ clips altogether in April and May. #### 72

Figure 2.6. GSM8K example: question, rationale with intermediate steps, and final answer.

Bibliography

- [1] Karl Cobbe et al. „*Training Verifiers to Solve Math Word Problems*”. 2021. arXiv: 2110.14168 [cs.LG].
- [2] Jason Wei et al. „*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*”. 2023. arXiv: 2201.11903 [cs.CL].
- [3] Xuezhi Wang et al. „*Self-Consistency Improves Chain of Thought Reasoning in Language Models*”. 2023. arXiv: 2203.11171 [cs.CL].
- [4] Edward J. Hu et al. „*LoRA: Low-Rank Adaptation of Large Language Models*”. 2021. arXiv: 2106.09685 [cs.CL].
- [5] Tim Dettmers et al. „*QLoRA: Efficient Finetuning of Quantized LLMs*”. 2023. arXiv: 2305.14314 [cs.LG].
- [6] Thomas Wolf et al. „*HuggingFace’s Transformers: State-of-the-art Natural Language Processing*”. 2020. arXiv: 1910.03771 [cs.CL].
- [7] Taku Kudo and John Richardson. „*SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing*”. 2018. arXiv: 1808.06226 [cs.CL].
- [8] Rico Sennrich, Barry Haddow, and Alexandra Birch. „*Neural Machine Translation of Rare Words with Subword Units*”. 2016. arXiv: 1508.07909 [cs.CL].
- [9] Qwen et al. „*Qwen2.5 Technical Report*”. 2025. arXiv: 2412.15115 [cs.CL].
- [10] DeepSeek-AI et al. „*DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*”. 2025. arXiv: 2501.12948 [cs.CL].
- [11] OpenRouter. „*OpenRouter: The Unified Inference for LLMs*”. <https://openrouter.ai>. Accessed 2025. 2023.
- [12] Ashish Vaswani et al. „*Attention Is All You Need*”. 2023. arXiv: 1706.03762 [cs.CL].
- [13] Jared Kaplan et al. „*Scaling Laws for Neural Language Models*”. 2020. arXiv: 2001.08361 [cs.LG].
- [14] Tom B. Brown et al. „*Language Models are Few-Shot Learners*”. 2020. arXiv: 2005.14165 [cs.CL].
- [15] Long Ouyang et al. „*Training language models to follow instructions with human feedback*”. 2022. arXiv: 2203.02155 [cs.CL].

- [16] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. „*Distilling the Knowledge in a Neural Network*”. 2015. arXiv: 1503.02531 [cs.CL].
- [17] Sergey Zagoruyko and Nikos Komodakis. „*Paying More Attention to Attention: Improving the Performance of Convolutional Neural Networks via Attention Transfer*”. 2017. arXiv: 1612.03928 [cs.CV].
- [18] Tommaso Furlanello et al. „*Born Again Neural Networks*”. 2018. arXiv: 1805.04770 [cs.LG].
- [19] Qizhe Xie et al. „*Self-training with Noisy Student improves ImageNet classification*”. 2020. arXiv: 1911.04252 [cs.LG].
- [20] Jianping Gou et al. „*Knowledge Distillation: A Survey*”. 2021. arXiv: 2006.05525 [cs.LG].
- [21] Yoon Kim and Alexander M. Rush. „*Sequence-Level Knowledge Distillation*”. 2016. arXiv: 1606.07947 [cs.CL].
- [22] Liunian Harold Li et al. „*Symbolic Chain-of-Thought Distillation: Small Models Can Also "Think" Step-by-Step*”. 2024. arXiv: 2306.14050 [cs.CL].
- [23] Neil Houlsby et al. „*Parameter-Efficient Transfer Learning for NLP*”. 2019. arXiv: 1902.00751 [cs.LG].
- [24] Brian Lester, Rami Al-Rfou, and Noah Constant. „*The Power of Scale for Parameter-Efficient Prompt Tuning*”. 2021. arXiv: 2104.08691 [cs.CL].
- [25] Xiang Lisa Li and Percy Liang. „*Prefix-Tuning: Optimizing Continuous Prompts for Generation*”. 2021. arXiv: 2101.00190 [cs.CL].
- [26] „bitsandbytes”. <https://github.com/bitsandbytes-foundation/bitsandbytes>.
- [27] NVIDIA Corporation. „*QLoRA Parameter-Efficient Fine-Tuning - NVIDIA NeMo Documentation*”. 2024. URL: https://docs.nvidia.com/nemo-framework/user-guide/24.09/sft_peft/qlora.html.
- [28] Quentin Lhoest et al. „*Datasets: A Community Library for Natural Language Processing*”. 2021. arXiv: 2109.02846 [cs.CL].